MODULE 20 · DOMAIN 3 - DEPLOYMENT AND ORCHESTRATION

Choosing an inference option

Real-time, serverless, asynchronous, batch. The highest-value decision in Domain 3.

MODULE 20 of 43 DOMAIN 3 - Deploy WEIGHT 24%

admin@sthithapragna.com

Who is waiting, and how long will they wait?

admin@sthithapragna.com

Four ways to get a prediction

01

Real-time

Persistent endpoint. Always on, answers immediately.

02

Serverless

No instances. Scales to zero, cold starts on wake.

03

Asynchronous

Queued. Large payloads, long processing, results to S3.

04

Batch

A job over a dataset. No endpoint at all.

ON THE EXAM

Two of these can scale to zero. Two of them bill continuously. That distinction drives most of the cost questions.

Three things exist behind every endpoint

OBJECT	WHAT IT HOLDS
Model	The artefact in S3, the container image, and the execution role
Endpoint configuration	Which model or models, instance type and count, or serverless config
Endpoint	The running thing with a URL. Points at one endpoint configuration
Why it matters	You update an endpoint by giving it a NEW configuration, not by editing one

ON THE EXAM

Endpoint configurations are immutable. That immutability is what makes blue/green and canary deployment possible in module 24.

Invoking an endpoint

- **InvokeEndpoint** for a normal request. You send the payload, you get the prediction.
- **InvokeEndpointWithResponseStream** streams the response back as it is produced.
- Streaming is why real-time allows **8 minutes** rather than 60 seconds: the connection stays useful.
- Content type and accept headers tell the container how to parse and what to return.
- For async you call **InvokeEndpointAsync** with an S3 location, not the payload itself.

> ON THE EXAM
>
> Response streaming matters most for generative models, where the first token can arrive in under a second and the last one takes a minute.

The published limits

OPTION	PAYLOAD	PROCESSING TIME
Real-time	up to 25 MB	60 seconds, or 8 minutes streaming
Serverless	up to 4 MB	up to 60 seconds
Asynchronous	up to 1 GB	up to one hour
Batch transform	datasets GBs in size	days

ON THE EXAM

Learn these four rows. Most inference questions are decided by one number in this table rather than by judgement.

Two of those limits recently increased

- **Real-time payload was 6 MB.** The current documentation says **25 MB**.
- **Asynchronous processing was 15 minutes.** The current documentation says **one hour**.
- Plenty of material still quotes the old figures, including some AWS pages.
- The 2021 asynchronous announcement still says 6 MB and 15 minutes, because it is an announcement.
- Answer from the inference options page: **25 MB and one hour**.

> ON THE EXAM
>
> This matters practically: a 9 MB payload used to rule out real-time inference, and no longer does.

Real-time inference

- A persistent, fully managed **REST endpoint**, backed by instances you choose.
- For **low latency or high throughput** with sustained traffic. Responses in milliseconds.
- Any instance type, including large GPU instances. Auto scaling from module 23.
- Three deployment modes: **single model, multi-model, inference pipeline**. Module 21.
- **It bills continuously**, whether it receives one request an hour or a thousand a second.

> ON THE EXAM
>
> The default answer when a human is waiting, and the default source of surprise bills when they are not.

When a cold start is the objection

- **Provisioned Concurrency** on serverless, for a warm baseline.
- A **real-time endpoint** with a minimum of one instance, which never sleeps.
- Keep-warm pings work and are a workaround, not a design. They also cost invocations.
- **Or accept it.** For an internal tool, two seconds once an hour is not a problem.
- Smaller containers and smaller models start faster, so model size affects cold start length.

> ON THE EXAM
>
> Ask who experiences the cold start and how often. A daily batch user will never notice; a customer on a checkout page will.

Serverless inference

- No instances to manage. AWS provisions on demand and **scales to zero** when idle.
- You pay for compute duration and requests, **not for idle time**.
- Configure **memory from 1 GB to 6 GB**, and a maximum concurrency.
- **Cold starts** add roughly 1 to 5 seconds after an idle period.
- **Provisioned Concurrency** removes cold starts for a baseline, and can itself be auto scaled.

> ON THE EXAM
>
> Intermittent or unpredictable traffic is the signal. If the endpoint would be idle most of the day, this is usually the answer.

Provisioned Concurrency on serverless

- Keeps a configured number of concurrent executions **warm and ready**.
- So the first request after idle does not pay the 1 to 5 second cold start.
- It has a **baseline cost**, because something is being kept warm. Serverless stops being free when idle.
- It can be scaled with **Application Auto Scaling**, so the warm baseline follows your traffic profile.
- A middle ground between pure serverless and a real-time endpoint.

> ON THE EXAM
>
> If you provision concurrency for all your traffic, you have rebuilt a real-time endpoint with extra steps. Use it for a baseline.

Real-time or serverless?

REAL-TIME, WHEN

Traffic is sustained

- Steady or high request volume.
- Cold starts are unacceptable.
- You need GPU, or a specific instance type.
- Payload up to 25 MB.

SERVERLESS, WHEN

Traffic is intermittent

- Sporadic or unpredictable requests.
- Long idle periods between bursts.
- A few seconds of cold start is tolerable.
- Payload under 4 MB, under 60 seconds.

THE TIEBREAKER

Serverless has no GPU option. If the model needs a GPU, serverless is ruled out regardless of the traffic pattern.

admin@sthithapragna.com

Asynchronous inference

- Queues incoming requests and processes them asynchronously, with an internal queue.
- Built for **large payloads and long processing**: up to 1 GB and up to one hour.
- **You upload the payload to S3 and send the endpoint a pointer.** Results land back in S3.
- Completion can notify through **SNS**, so the caller does not poll.
- **Scales instance count to zero** when there are no requests to process.

> ON THE EXAM
>
> The S3 pointer is why the payload limit is so much larger: the model never receives a huge HTTP request.

Asynchronous changes your application

- **It is not a synchronous call.** The response is an acknowledgement, not a prediction.
- Your application must poll for the result, or handle an SNS notification.
- That is real work, and it is the main reason teams avoid it when they should not.
- In exchange: large payloads, long processing, scale to zero, and no request timeouts.
- Each request gets an identifier you can use to track its status.

> ON THE EXAM
>
> Async is the option people forget, because adopting it means changing the client as well as the endpoint.

What the queue gives you, and its limits

ASPECT	BEHAVIOUR
Internal queue	Requests are queued, so a burst does not fail. It absorbs spikes
Concurrency	You control how many requests are processed at once per instance
Success and failure paths	Separate S3 locations for successful output and for failures
Notification	SNS topics for success and failure, so the caller need not poll
Scale to zero	Auto scaling to zero instances when the queue empties
Waking up	A queued request triggers scale-up, which takes time on the first call

ON THE EXAM

The queue is the real feature. A traffic spike that would return 429s on a real-time endpoint simply queues here.

Batch transform

- A **job**, not an endpoint. It starts, processes a dataset from S3, writes results, and stops.
- **No idle charge at all**, because the instances terminate when the job completes.
- Handles datasets that are gigabytes in size, with processing times measured in days.
- Splits input into **mini-batches**, with MaxPayloadInMB up to 100 MB each.
- Also used for preprocessing a dataset, not only for scoring one.

> ON THE EXAM
>
> The cheapest option by a wide margin for offline work, because you pay only for the seconds the job runs.

The settings that decide batch performance

SETTING	EFFECT
`InstanceCount`	How many instances process the dataset in parallel
`MaxPayloadInMB`	Size of each mini-batch sent to the container. Up to 100 MB
`BatchStrategy`	MultiRecord packs many records per request; SingleRecord sends one
`SplitType`	How the input is divided: by line, by RecordIO, or not at all
`AssembleWith`	How output records are reassembled into the result file
`JoinSource`	Attach input columns to the output, so predictions keep their identifiers

ON THE EXAM

JoinSource is the one people discover too late: without it, you get predictions in order with no key to join them back on.

Asynchronous or batch transform?

ASYNCHRONOUS, WHEN

Requests arrive one at a time

- Somebody submitted something and wants it back.
- Requests arrive unpredictably through the day.
- Near real-time: minutes, not tomorrow.
- The endpoint persists, and scales to zero.

BATCH TRANSFORM, WHEN

The whole dataset exists already

- All the data is available up front.
- It runs on a schedule, or ad hoc.
- Nobody is waiting for an individual result.
- No endpoint exists at all.

THE TIEBREAKER

Both are 'not synchronous'. The difference is whether requests arrive individually, or the dataset is sitting there waiting.

admin@sthithapragna.com

Signal to inference option

WHAT THE SCENARIO SAYS	THE OPTION
A user is waiting, milliseconds matter	Real-time
Sustained traffic, GPU needed	Real-time
Sporadic traffic, long idle periods	Serverless
Payload over 4 MB, or over 60 seconds	Not serverless
Payload over 25 MB	Asynchronous or batch
Processing takes more than a minute	Asynchronous or batch
Large files submitted through the day	Asynchronous
Nightly scoring of a whole dataset	Batch transform
Nobody is waiting for an individual result	Batch transform

ON THE EXAM

This is the slide to revise. Nine rows, and most inference questions are one of them.

Bedrock or a SageMaker AI endpoint?

BEDROCK, WHEN

You consume a model

- A managed model behind one API.
- Per-token billing, no infrastructure.
- Guardrails, Knowledge Bases, Evaluations.
- The default for language tasks.

SAGEMAKER AI ENDPOINT, WHEN

You control the model

- An open model from JumpStart.
- A model you trained or heavily customised.
- Specific instance types, including Inferentia.
- You need the artefact and the container.

THE TIEBREAKER

This is module 01's fork again, seen from the deployment side. The same question decides both.

Sometimes you do not need an endpoint at all

- A small scikit-learn or XGBoost model can run **inside a Lambda function**.
- No endpoint, no idle cost, and latency in milliseconds once warm.
- Constrained by Lambda's package size, memory and 15-minute limit.
- You now own the serving code, the dependencies and the model loading.
- Reasonable for a tiny model at low volume. Not a general pattern.

ON THE EXAM

Worth knowing it exists, so you recognise it as a legitimate option rather than assuming an endpoint is compulsory.

Testing an endpoint

- Deploy to a **non-production endpoint** first and invoke it with real payloads.
- Check the container handles your content type, and returns what the caller expects.
- **Load test it.** Latency at one request per second tells you nothing about latency at a thousand.
- **Inference Recommender** can run load tests across instance types for you. Module 22.
- Serialisation mismatches are the most common deployment failure, and they are trivial to catch here.

> ON THE EXAM
>
> The model was tested in module 17. The endpoint is a different thing, and it has its own failure modes.

Resilience of an endpoint

- SageMaker AI distributes endpoint instances across **Availability Zones** automatically.
- **A single-instance endpoint has no redundancy.** Two instances is the minimum for production.
- Health checks replace unhealthy instances without you intervening.
- For Region-level resilience you need a second endpoint in another Region, and routing in front.
- Serverless and async handle instance failure for you, which is part of what you are buying.

ON THE EXAM

One instance is a development configuration. It is also the configuration most production endpoints are accidentally left in.

The three numbers to look for

- **Payload size.** 4 MB and 25 MB are the two thresholds that eliminate options.
- **Processing time.** 60 seconds and one hour are the next two.
- **Traffic pattern.** Sustained, sporadic, or a scheduled batch.
- Scenarios plant these deliberately. A stated file size is never decoration.
- Find all three before comparing options, and usually only one remains.

> ON THE EXAM
>
> Payload size, processing time, traffic pattern. Read for those three and the answer falls out almost mechanically.

Near real-time is not real-time

- **'Near real-time'** in AWS documentation describes **asynchronous** inference.
- It means results in minutes, not milliseconds. The request is queued, not answered.
- So a scenario saying 'near real-time' is usually pointing at async, not at real-time.
- Read the latency requirement in the scenario rather than trusting the adjective.
- 'Interactive', 'while the user waits' and 'sub-second' all mean real-time.

ON THE EXAM

One word, two very different options. AWS uses 'near real-time' specifically for the asynchronous inference use case.

Using more than one option

- A real-time endpoint for interactive traffic, plus batch transform for a nightly backfill.
- Serverless for a development environment, real-time for production.
- **The same model artefact serves all four**, so switching is a deployment change, not a rebuild.
- Batch transform is also useful for generating predictions to evaluate, from module 17.
- Two options means two things to monitor and two cost lines. Do not split without a reason.

ON THE EXAM

The portability is the useful part: the option is a deployment decision, and it can be revisited cheaply.

Quotas worth knowing about

QUOTA	NOTE
Endpoints per account	A cap on how many can exist at once. Raisable
Instances per endpoint	Per instance type, and it constrains auto scaling headroom
Accelerated instance quotas	Often zero by default. Request before you need them
Serverless concurrency	Maximum concurrency per endpoint, and per account
Batch transform instances	Constrains how far a job can parallelise

ON THE EXAM

An auto scaling policy that cannot exceed your instance quota will silently stop scaling at the quota, not at the policy maximum.

How each option bills

OPTION	COST SHAPE
Real-time	Per instance-hour, continuously, until you delete the endpoint
Serverless	Per compute duration and per request. Nothing while idle
Serverless with Provisioned Concurrency	A baseline charge, plus usage. Cold starts removed
Asynchronous	Per instance-hour while processing. Zero when the queue is empty
Batch transform	Per instance-second for the job only. No idle charge ever

ON THE EXAM

An idle real-time endpoint is the single most common waste in an ML account. It bills all night and all weekend.

Data capture

- **Data capture** records inference requests and responses from an endpoint to S3.
- That captured data is what **drift monitoring** compares against a baseline. Module 29.
- You set a **sampling percentage**, so you need not capture everything at high volume.
- It is **off by default**. Enable it when you deploy, not when you need it.
- Captured payloads inherit the sensitivity of the data. Module 08 applies.

> ON THE EXAM
>
> Without data capture there is no record of what the model was actually asked in production, so drift cannot be measured at all.

Symptoms, and what they usually mean

SYMPTOM	USUAL CAUSE
Requests failing on large inputs	Payload above the option's limit. Check 4 MB and 25 MB
Intermittent multi-second latency	Serverless cold starts after idle periods
Endpoint times out on long inference	Over 60 seconds. Async or batch instead
Large bill, very few predictions	Real-time endpoint idle most of the time
Async endpoint slow to first result	Scaled to zero. It is starting instances
Batch job slower than expected	Too few instances, or mini-batch size badly set
Model needs a GPU on serverless	Not supported. Serverless is CPU only

ON THE EXAM

Rows two and five are the same mechanism seen from both sides: scale-to-zero saves money and costs a wake-up.

admin@sthithapragna.com

Match the option to the **shape of the traffic**, not to how important the model is.

admin@sthithapragna.com

A media company runs a model that analyses uploaded video files. Files average 200 MB and inference takes around twelve minutes per file. Editors upload files throughout the working day, unpredictably.

Editors expect results within a few minutes of processing finishing, and are content to be notified rather than wait on a connection.

Which inference option fits?

A. Real-time endpoint on a large GPU instance

B. Serverless inference with 6 GB memory and provisioned concurrency

C. Asynchronous inference, with results delivered to S3 and an SNS notification

D. Batch transform, run hourly over the files uploaded that hour

◆ PAUSE AND DECIDE

Which inference option fits?

A Real-time endpoint on a large GPU instance

200 MB exceeds the 25 MB payload limit, and twelve minutes exceeds the 60-second timeout. It fails on both numbers.

B Serverless inference with 6 GB memory and provisioned concurrency

4 MB payload and 60-second limits, and no GPU option. It fails more comprehensively than real-time does.

C Asynchronous inference, with results delivered to S3 and an SNS notification

Up to 1 GB and up to one hour covers both numbers. Requests arrive individually, and SNS matches the stated willingness to be notified.

D Batch transform, run hourly over the files uploaded that hour

Would work, and it adds up to an hour of waiting before processing even starts. Editors want results shortly after their own upload.

THE SIGNAL

Two numbers eliminate two options outright: 200 MB against a 25 MB limit, and twelve minutes against 60 seconds.

An internal HR tool classifies expense descriptions into categories. It receives roughly 40 requests on a typical day, clustered around month end, and none at all overnight or at weekends.

Payloads are a few kilobytes and inference takes under a second. A two-second delay on the first request after a quiet period is acceptable to the team.

Which option should they deploy on?

A Serverless inference

B Real-time inference on a small CPU instance with auto scaling

C Asynchronous inference, to scale to zero between requests

D Batch transform, run every fifteen minutes

◆ PAUSE AND DECIDE

Which option should they deploy on?

A Serverless inference

Sporadic traffic, long idle periods, tiny payloads, sub-second inference, and a stated tolerance for cold starts. Every condition matches.

B Real-time inference on a small CPU instance with auto scaling

Auto scaling has a floor of one instance, so it bills all night and all weekend for 40 requests a day. It cannot scale to zero.

C Asynchronous inference, to scale to zero between requests

Scales to zero, and forces the client to poll or handle SNS for a sub-second classification. Complexity with no benefit here.

D Batch transform, run every fifteen minutes

Turns an interactive tool into one with up to fifteen minutes of latency. A user is waiting, even if only briefly.

THE SIGNAL

The sentence about tolerating a two-second delay is permission to use serverless. Without it, cold starts would be the objection.

> A bank scores 30 million customer records for propensity to churn, once a week. The scores are loaded into a marketing database on Sunday night and used throughout the following week.
>
> Each record is a few hundred bytes. Nobody consumes an individual score before the load completes.

What should they use?

A. A real-time endpoint invoked in a loop by a Lambda function

B. Serverless inference with maximum concurrency raised

C. Asynchronous inference, submitting all 30 million records

D. Batch transform, run weekly against the dataset in S3

◆ PAUSE AND DECIDE

What should they use?

A A real-time endpoint invoked in a loop by a Lambda function

30 million individual HTTP calls against a persistent endpoint that bills all week for one night's work. Module 01's scenario, at larger scale.

B Serverless inference with maximum concurrency raised

Removes the idle cost and keeps the wrong shape: 30 million separate invocations, with cold starts, to produce one file.

C Asynchronous inference, submitting all 30 million records

Built for individual large payloads arriving over time, not for one enormous dataset available up front. Wrong tool, right instinct about latency.

D Batch transform, run weekly against the dataset in S3

The whole dataset exists up front, nobody waits for an individual score, and instances terminate when the job ends. No idle charge at all.

THE SIGNAL

Nobody is waiting, and the dataset already exists. Those two facts together are batch transform, every time.

A team needs to deploy a document classification model behind a synchronous API. Documents average 9 MB and inference takes about 4 seconds. Traffic is steady through the working day.

An engineer states that real-time inference is impossible because the payload exceeds the 6 MB limit, and proposes an asynchronous endpoint with client polling instead.

How should you respond?

A. Agree. 9 MB exceeds the real-time payload limit, so asynchronous is required

B. Correct the figure. The real-time payload limit is now 25 MB, so real-time works

C. Propose serverless inference, which has a higher payload limit than real-time

D. Propose compressing documents below 6 MB before sending them

◆ PAUSE AND DECIDE

How should you respond?

A. Agree. 9 MB exceeds the real-time payload limit, so asynchronous is required

Accepts a stale figure. It also forces the client into polling for a four-second inference behind a synchronous API.

B. Correct the figure. The real-time payload limit is now 25 MB, so real-time works

The current limit is 25 MB, and 4 seconds is well inside the 60-second timeout. Steady traffic behind a synchronous API is real-time's case exactly.

C. Propose serverless inference, which has a higher payload limit than real-time

Backwards. Serverless is 4 MB, the lowest of all the options, so 9 MB fails there more decisively.

D. Propose compressing documents below 6 MB before sending them

Engineering work to satisfy a limit that no longer exists, and it risks losing information from the documents.

THE SIGNAL

The 6 MB figure was correct for years and is now stale. Old limits outlive their accuracy in documentation, blog posts and memory alike.

What to take from this module

- **Real-time 25 MB and 60 s. Serverless 4 MB and 60 s. Async 1 GB and one hour. Batch, no practical limit.**
- **6 MB and 15 minutes are the old figures.** Recognise them as a sign of a stale source.
- **Read for three things:** payload size, processing time, traffic pattern.
- **Serverless has no GPU.** Real-time and async bill differently: continuous against scale-to-zero.
- **Async is for requests arriving. Batch is for a dataset already sitting there.**

> ON THE EXAM
>
> Module 21 covers what you put behind a real-time endpoint: multi-model, multi-container and inference pipelines.